KLE Technological University, Hubballi

Computer Science & Engineering (AI)

Agentic AI (26ECAC401) — Lab Worklet Review & Evaluation Sheet

14 Hands-on Labs · to be filled by the student team and marked by the reviewer

Team name / number	InternTrack Team 11
Worklet title	InternTrack — AURA-Powered Internship Application & Tracking Portal
KPI & Expectations	Build one unified internship workflow while demonstrating all 14 Labs: planning, tools, reusable skills, memory, governed connector, controlled runtime, LangGraph workflow, parallel swarm, deep research & grounding, cloud ops resilience & self-healing, agentic SDLC factory, safety governance & rollback, discipline specialists, and capstone team-owned deployment.
Team members (Name — USN)	Prerna A -01fe23bci037 Aishwarya B A -01fe23bci095 Sonu Vaishnavi -01fe23bci093 Nischita N -01fe23bci106
Review (Week / date)	Week 8 / 30-09-2026

How each lab is marked (max 10 per lab)

- **Concept (2) —** Is the lab's concept correctly understood and clearly mapped to the team's worklet?
- **Technical stack (2) —** Is the chosen stack appropriate and correctly justified for the task?
- **Implementation (4) —** Does the implementation work and correctly realise the concept (loop / tool / skill / graph / etc.)?
- **Explanation & demo (2) —** Is the explanation clear and supported by evidence — repo, screenshots, or a live demo?

Instructions: *The team fills the four shaded description rows for every lab (be specific; attach evidence). The reviewer fills the marks strip and remarks. Total = sum of 14 labs (out of 140); scale to the lab component weightage as required.*

Lab 1 — Agent vs Chatbot

Concept: Agent loop & the plan–act–observe–revise cycle (vs a stateless chatbot) · Reference worklet: study planner that clarifies, plans and revises

1. Concept applied in your worklet — what you used and where	Used the agent loop for InternTrack's AURA planner: clarify the vague internship request, create a structured plan, receive feedback, and revise the plan.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Python 3.10+, AURA planner module, REST backend, SQLite, and the InternTrack web UI. Session state carries the clarification/plan context.
3. Implementation — what you built and how it works (key steps / architecture)	Flow: user request → clarification questions → initial structured plan → user feedback → revised plan. The planner output is shown in the 14-Lab Studio.
4. Evidence — repo link / screenshots / demo / sample output	Live 14-Lab Studio demo with clarification, initial plan, and revised plan; implementation: aura/lab1_planner.py and Lab 1 execution trace.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 2 — Tool-Using Agent

Concept: Tool integration & function calling · Reference worklet: calculator, file-reader and a mock-API tool

1. Concept applied in your worklet — what you used and where	Used typed tools so AURA obtains student, opportunity, application/status, eligibility and vacancy facts from tools instead of manually asserting database values.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Python typed tool layer, SQLite-backed workflow/API functions, AURA runtime, schema validation, and the InternTrack Lab Studio UI with tool-call trace.
3. Implementation — what you built and how it works (key steps / architecture)	A request is mapped to a typed tool, validated, executed against real data, and returned as structured output. Tool calls and results are visible in the execution trace.
4. Evidence — repo link / screenshots / demo / sample output	Live Lab 2 execution showing tool calls and real database results; evidence includes the 14-Lab Studio tool trace and acceptance tests.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 3 — Skill Creation

Concept: Reusable skills (instructions + tools + resources) · Reference worklet: a reusable research-summary skill

1. Concept applied in your worklet — what you used and where	Packaged repeated internship capabilities as reusable schema-typed skills: InternshipPlanSkill, ApplicationValidatorSkill, OpportunityMatchingSkill, and DocumentFormatterSkill.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Python skill modules, typed input/output schemas, AURA skill executor, SQLite-backed data/workflows, and the Lab Studio skill selector.
3. Implementation — what you built and how it works (key steps / architecture)	Each selected skill accepts structured input, validates its schema, performs its defined operation, and returns a standardized verified output that can be reused by the agent workflow.
4. Evidence — repo link / screenshots / demo / sample output	Live Lab 3 Skill Creation demo showing the reusable skill selector and execution output for Rahul Kumar; evidence in the Lab 3 Studio.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 4 — Memory & Retrieval

Concept: Short-term state vs long-term retrieval (RAG) · Reference worklet: session state vs a stored/retrieved reference

1. Concept applied in your worklet — what you used and where	Separated AURA's short-term working SessionState from persistent historical submission/reference memory. Used session context for the active plan and history for prior internship information.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Python session/state handling, SQLite persistent memory records, keyword/cosine-style retrieval, and the InternTrack Lab 4 memory UI.
3. Implementation — what you built and how it works (key steps / architecture)	Active clarification answers and revised plan remain in working memory, while prior submissions/preferences are retrieved from persistent history; opportunity similarity search is also demonstrated.
4. Evidence — repo link / screenshots / demo / sample output	Live Lab 4 demo showing SessionState, historical records, persistent memory records, and cosine-similarity opportunity results.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 5 — MCP-Style Connector

Concept: Model Context Protocol & connectors · Reference worklet: one server exposing File + SQLite + API

1. Concept applied in your worklet — what you used and where	Implemented a governed MCP-style connector as the controlled gateway between AURA and selected data sources. Student access is scoped to the caller; coordinator/HOD access is broader.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Python MCPConnectorServer, HTTP REST API, Bearer authentication, HMAC-SHA256 token validation, role-based authorization, SQLite workflow layer, and access telemetry.
3. Implementation — what you built and how it works (key steps / architecture)	Three connector endpoints expose student profiles, published opportunities, and application feeds. Missing/invalid auth returns 401; unauthorized cross-student access returns 403; successful calls return live DB data.
4. Evidence — repo link / screenshots / demo / sample output	Live Lab 5 demo: authenticated self-access returns 200, cross-student access returns 403, anonymous access returns 401. Dedicated Lab 5 governance tests: 12/12 passed.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 6 — OpenCLA / OpenClaw Runtime

Concept: Agent runtime with logging · Reference worklet: runtime running skills, tools, memory + an audit log

1. Concept applied in your worklet — what you used and where	Used a controlled agent runtime with budgets, step tracking, checkpoints, audit events, and a mandatory Coordinator Human Gate before consequential internship approval/rejection.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Python ControlledAgentRuntime, SQLite audit_logs, workflow/submission APIs, REST approval/rejection endpoints, role checks, and the AURA Lab Studio UI.
3. Implementation — what you built and how it works (key steps / architecture)	Safe eligibility/vacancy checks run first. A risky action pauses with PAUSED_FOR_APPROVAL; only an authorized Coordinator can approve or reject. Approval/rejection then commits through the real submission workflow.
4. Evidence — repo link / screenshots / demo / sample output	Live Lab 6 Human Gate demo plus database evidence before/after approval and rejection. Dedicated verification passed all 13 criteria; Labs 1–5 regression passed.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 7 — Agentic Node Graph

Concept: Graph-based orchestration (LangGraph) · Reference worklet: campus app graph: requirement → design → release

1. Concept applied in your worklet — what you used and where	Modelled the internship application workflow as a LangGraph StateGraph: ingest → planner → discovery → eligibility → application → validation → human gate → publish, with bounded retry.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Official LangGraph StateGraph, TypedDict state, MemorySaver checkpointing, SQLite workflows, Python backend/API, and InternTrack Human Gate UI.
3. Implementation — what you built and how it works (key steps / architecture)	Validation conditionally routes to pass, bounded retry, or escalation. The graph pauses at the Coordinator gate using a checkpoint; approval resumes the same graph and publish commits the canonical workflow.
4. Evidence — repo link / screenshots / demo / sample output	Live Lab 7 LangGraph Studio and verification suite: real StateGraph, checkpoint pause/resume, Coordinator-only approval, rejection reason, bypass protection, and 9/9 Lab 7 tests passed.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 8 — Parallel Swarm

Concept: Parallel execution, swarms & fault-tolerant merge · Reference worklet: hackathon planner: 4 role agents + a merge step

1. Concept applied in your worklet — what you used and where	Used a parallel swarm to evaluate up to four internship preferences concurrently, merge all worker results, rank opportunities, and then wait for Coordinator approval.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Official LangGraph StateGraph, MemorySaver, parallel Pregel workers, Annotated list reducer with operator.add, SQLite eligibility/vacancy checks, REST API, and Human Gate UI.
3. Implementation — what you built and how it works (key steps / architecture)	split → 4 parallel workers → fan-in reducer → eligibility/vacancy ranking → Coordinator gate. Worker failures are isolated as WORKER_ERROR; approval resumes the same checkpoint into the commit node.
4. Evidence — repo link / screenshots / demo / sample output	Live Lab 8 demo shows four parallel branches, real DB eligibility/vacancy data, ranking, and Coordinator Human Gate. Dedicated Lab 8 suite: 13/13 passed; Labs 1–7 regression passed.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 9 — Deep Research Agent

Concept: Deep research with evidence synthesis · Reference worklet: AI topic briefing with a cited evidence table

1. Concept applied in your worklet — what you used and where	Applied Deep Research & Grounding (Phase C: Applied Reliability) to build an auditable evidence record for every student internship candidate output. Rather than taking LLM generation or student claims on trust, the agent performs grounded verification against authoritative in-scope sources: the faculty-provided evaluation rubric (SEC-1.1 through SEC-2.3), institutional student registry, and the official catalog of vetted opportunities/MoUs. Strictly enforces the Build Book pitfall avoidance: plausible-sounding but ungrounded entities are rejected, ensuring zero hallucinations slip through to validation.
2. Technical stack — languages, frameworks, libraries, APIs, models, tools	Python 3.14, Official LangGraph StateGraph (candidate_generation -> deep_research_grounding -> validation_check), TypedDict GroundingGraphState, SHA-256 cryptographic evidence hashing, SQLite3 (opportunity catalog & student registry), REST API (POST /api/aura/lab/9/run), HTML5 / Vanilla CSS3 / JavaScript (AURA Studio Evidence Table Viewer & Adversarial Test Harness).

3. Implementation — what you built and how it works (key steps / architecture)	Implemented aura/lab9_grounding.py following the exact Build Book grounding loop: for c in candidates: ev = research(c, source='the faculty-provided test cases and rubric') if not ev.in_scope: reject(c) c.evidence = ev Architecture: 1. Candidate Generation Node: Resolves candidate companies from student submission preferences or benchmark inputs. 2. Deep Research Grounding Node: Inspects each candidate against catalog records and student metrics. Attaches a unit-level EvidenceRecord (containing claim, citation, evidence text, rule cited, verification hash) and aggregates into an application-level EvidenceTable. 3. Strict Adversarial Defense: Plausible-sounding hallucinated companies (e.g., 'Quantum Apex AI', 'CyberPhantom Blockchain Labs') without departmental MoUs receive in_scope=False and are immediately marked REJECTED_UNGROUNDED. 4. Validation Node: Acts as final gate verifying that 100% of admitted outputs map to cited in-scope evidence.
4. Evidence — repo link / screenshots / demo / sample output	1. Core Module: aura/lab9_grounding.py (EvidenceRecord, EvidenceTable, LangGraph StateGraph, run_lab9_deep_research). 2. Dedicated Test Suite: test_lab9_grounding.py -- 8/8 tests PASS verifying rubric integrity, citation accuracy, ungrounded entity rejection, and REST API contract. 3. Full Acceptance Regressions: tests/test_acceptance.py (13/13 PASS) & test_lab8_complete_suite.py (13/13 PASS) verifying zero breakage across Labs 1-8. 4. Live REST Endpoint: POST /api/aura/lab/9/run (tested with standard & adversarial modes). 5. AURA Studio Web UI: Lab 9 tab featuring interactive evidence table, grounding percentage meter, per-unit citation drawer, and adversarial hallucination injection toggle.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 10 — Cloud Operations Agent

Concept: Cloud agents, observability & failure handling · Reference worklet: diagnose and fix a failed deployment

Applied Agentic Cloud Operations & Self-Healing Runtime (Phase C: Applied Reliability). Rather than allowing transient network spikes, corporate API throttling, or database lock contentions to crash student applications, the agent implements automated fault diagnosis, error pattern classification, retry budget management, and graceful fallback to local cached catalogs. Enforces the Build Book pitfall avoidance: circuit breaker isolation prevents cascading service outages across the department.	
Python 3.14, aura/lab10_resilience.py, Exponential Backoff with Decorrelated Jitter, Circuit Breaker Pattern, Dead Letter Queue (DLQ), SQLite3 transaction rollback, REST API (POST /api/aura/lab/10/run), AURA Studio Interactive Chaos Simulator.	
Built ResilienceEngine and SelfHealingRuntime in aura/lab10_resilience.py following the Build Book ops loop: diagnose -> isolate -> retry_with_backoff -> fallback_to_cache -> record_dlq Key capabilities: 1. Fault Interception: Catches simulated DB lockups, external job board HTTP 500s, and socket timeouts. 2. Automated Healing: Executes up to 3 exponential retries with jitter; if exhausted, gracefully falls back to verified offline opportunity cache. 3. DLQ Logging: Unrecoverable failures generate a structured incident report in the dead-letter queue with incident ID and resolution recommendations.	

1. Core Module: aura/lab10_resilience.py (ResilienceEngine, CircuitBreaker, DeadLetterQueue). 2. Dedicated Test Suite: test_lab10_resilience.py -- 9/9 PASS (100%) verifying retry budgets, circuit tripping, fallback activation, and REST API. 3. Regression Suite: 100% PASS across Labs 1-9. 4. Live Demo: AURA Studio Lab 10 chaos test panel with real-time error injection and self-healing log stream.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 11 — Agentic SDLC Factory

Concept: Agentic SDLC (requirements → tested code) · Reference worklet: requirement → tested implementation pipeline

Applied Agentic SDLC Factory (Phase D: Governance & Capstone). Implements an automated software engineering pipeline that translates internship policy specifications directly into validated code. Enforces the Build Book 'Green Acceptance Gate' invariant: generated patches are run in an isolated test harness and can NEVER be deployed or merged unless 100% of acceptance tests pass.
Python 3.14, aura/lab11_sdlc_factory.py, AST (Abstract Syntax Tree) code validation, Python unittest isolated test harness, Automated Patch & Diff Generator, SQLite audit logging, REST API (POST /api/aura/lab/11/run), AURA Studio SDLC Factory Dashboard.
Built AgenticSDLCFactory in aura/lab11_sdlc_factory.py implementing the Build Book pipeline: spec_ingestion -> test_generation -> candidate_patch_generation -> sandbox_execution -> green_gate_verification Key steps: 1. Specification Ingestion: Parses regulatory rules (e.g., minimum CGPA 8.0, 4-month minimum duration). 2. Test-Driven Sandbox: Executes test cases against candidate implementations in an isolated execution harness. 3. Self-Refinement: If a test fails, the agent inspects the stack trace and refines the code diff (up to 3 iterations). 4. Green Gate Enforcer: Only code passing 100% of test assertions receives the green gate sign-off.
1. Core Module: aura/lab11_sdlc_factory.py (AgenticSDLCFactory, GreenGateVerifier, SandboxExecutor). 2. Dedicated Test Suite: test_lab11_sdlc_factory.py -- 9/9 PASS (100%) verifying spec parsing, patch execution, regression gating, and REST API. 3. Regression Verification: 100% PASS across Labs 1-10. 4. Live Demo: AURA Studio Lab 11 dashboard demonstrating code diff generation and green gate confirmation.

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 12 — Safety & Governance

Concept: Approval gates, audit logs, evaluation, rollback · Reference worklet: risk, approval, evaluation and rollback plan

Applied Safety & Governance (Phase D: Governance & Capstone). Formalizes human oversight, role-based access control (RBAC), evaluation metrics, and one-click reversible rollback. Adheres to the core course principle: 'Agents draft and flag; a named human approves.' Enforces strict confidentiality masking so students cannot view sensitive examiner rubric commentary or corporate NDA stipend clauses.	
Python 3.14, aura/lab12_governance.py, RBAC Permissions Matrix (ROLE_PERMISSIONS), SHA-256 cryptographic version hashing, SQLite governance_releases and governance_metrics tables, REST API (POST /api/aura/lab/12/run), AURA Studio Governance Desk.	
Built governance engine in aura/lab12_governance.py implementing: require actor.can_approve and out.status == 'draft' final = apply(human.edits, out); final.status = 'approved' governance.publish(final, version=v+1); metrics.record(...) on rollback(v): restore(version=v) Key mechanisms: 1. Hard-Stop Human Gate: Candidate draft evaluations remain in DRAFT status until an authorized Coordinator or HOD signs off. 2. Human Edits: Evaluators can override individual rubric sub-scores (Technical Depth, System Design, Viva). 3. Versioned Immutable Release: Publishes version v+1 with SHA-256 version hash. 4. Confidentiality Masking: Redacts sensitive faculty notes with [CONFIDENTIAL — REDACTED FOR STUDENT ROLE] for student callers. 5. 1-Click Rollback: Restores previous version v cleanly within milliseconds, marking the active release as superseded.	
1. Core Module: aura/lab12_governance.py (ROLE_PERMISSIONS, mask_confidential_content, publish_release, rollback_to_version). 2. Dedicated Test Suite: test_lab12_governance.py -- 9/9 PASS (100%) verifying approval gate, unauthorized student rejection, rubric overrides, masking, and rollback. 3. Verified Live on localhost:8000: Tested Coordinator approval, rollback, and student confidentiality masking. 4. Live Demo: AURA Studio Lab 12 governance controls.	

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 13 — Discipline-Specific Agents

Concept: Domain worklet for the team's discipline · Reference worklet: an ME / CE / EE domain-specific agent

Applied Discipline-Specific Agents (Phase D: Governance & Capstone). Integrates pluggable discipline specialist agents that enforce domain-specific engineering invariants for Computer Science / AI & Engineering internships. Evaluates technical work against rigorous code invariants rather than generic natural language summaries.	
Python 3.14, aura/lab13_discipline_specialists.py, AST static code analysis, 5 Pluggable Specialist Domain Packs (Python AI/ML, C Systems/POSIX, SQL DBMS, Java Enterprise, Data Structures), REST API (POST /api/aura/lab/13/run), AURA Studio Specialist Selector.	

Built DisciplineSpecialistManager in aura/lab13_discipline_specialists.py. Evaluates student project submissions against strict domain invariants: 1. Python AI/ML Specialist: Enforces PEP 484 type annotations, PyTorch FP16 KV-cache tensor invariants, and vectorized operations. 2. C Systems / POSIX Specialist: Verifies Valgrind memory leak cleanliness, mutex locking discipline, and thread-safe POSIX system calls. 3. SQL DBMS Specialist: Enforces 3NF normalization, covering B-Tree indexes, and transaction isolation. 4. Java Enterprise Specialist: Validates Spring Boot 3 IoC container design and @Transactional boundaries. 5. Data Structures Specialist: Verifies algorithmic complexity (e.g. O(N^2) Gale-Shapley stability proof). Submissions violating invariants receive targeted compiler diagnostics and deductions.	
1. Core Module: aura/lab13_discipline_specialists.py (DisciplineSpecialistManager, 5 Domain Specialist Packs). 2. Dedicated Test Suite: test_lab13_discipline_specialists.py -- 9/9 PASS (100%) verifying all 5 specialist packs, invariant violations, and REST API. 3. Full Regression Suite: 100% PASS across Labs 1-12. 4. Live Demo: AURA Studio Lab 13 interactive domain selector and invariant diagnostic feed.	

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Lab 14 — Capstone Project Studio

Concept: End-to-end, team-owned agentic system · Reference worklet: the full integrated worklet (all concepts)

Applied Capstone Project Studio (Phase D: Governance & Capstone). Integrates all 14 labs into a fully deployed, team-owned, end-to-end operational internship portal. Connects the complete lifecycle: student preference application -> AI candidate screening -> Coordinator approval gate -> ongoing 3-4 month progress tracking -> specialist evaluation -> HOD 12-credit sanction -> immutable audit trail.	
Complete Production InternTrack Stack: Backend: Python 3.14, LangGraph StateGraph engine, SQLite3 database (interntrack.db). Frontend: Vanilla HTML5, Modern CSS3 (Dark Theme, Glassmorphism, Micro-animations), Vanilla JavaScript ES6+. APIs: 15 REST endpoints covering applications, reviews, tracking, evaluations, audit, and 14-Lab Studio. Testing: 81 Automated Acceptance Tests across Labs 1 to 14.	
Built aura/lab14_capstone.py and fully integrated portal in server.py, static/index.html, app.js, and workflow.js. End-to-end integration: 1. Connector + Runtime: Scoped Bearer token access (Lab 5) with controlled runtime and step budgets (Lab 6). 2. Node Graph + Swarm: LangGraph stateful graph (Lab 7) with parallel multi-preference swarm matching (Lab 8). 3. Grounding + Resilience: Rubric evidence grounding (Lab 9) and self-healing error recovery (Lab 10). 4. SDLC + Governance: Green acceptance gate (Lab 11), human gate sign-off, masking, and 1-click rollback (Lab 12). 5. Specialists + Capstone: Technical code invariant evaluation (Lab 13) and single-session faculty evaluation dashboard (Lab 14).	

1. Core Module: aura/lab14_capstone.py (CapstonePipelineEngine, FacultyExaminerDashboard). 2. Dedicated Test Suite: test_lab14_capstone.py -- 10/10 PASS (100%). 3. Master 14-Lab Verification Runner: run_master_verification_1_to_14.py -- 81/81 PASS (100% PASS RATE). 4. Live Production Web Portal: Running and demonstrated live on http://localhost:8000 with real-world 65-student cohort, 50 company listings, and active HOD credit clearance.	

Concept (2)	Stack (2)	Implementation (4)	Explanation & demo (2)	Total (10)	Remarks

Marks Summary

Lab	Title	Max	Marks awarded
1	Agent vs Chatbot	10	
2	Tool-Using Agent	10	
3	Skill Creation	10	
4	Memory & Retrieval	10	
5	MCP-Style Connector	10	
6	OpenCLA / OpenClaw Runtime	10	
7	Agentic Node Graph	10	
8	Parallel Swarm	10	
9	Deep Research Agent	10	
10	Cloud Operations Agent	10	
11	Agentic SDLC Factory	10	
12	Safety & Governance	10	
13	Discipline-Specific Agents	10	
14	Capstone Project Studio	10	
	Total	**140**	

Reviewer signature: _____ Date: _____